EXPERIMENT.NO: 1 **hotel reservation system**

Draw a UML diagram for hotel reservation system. In a hotel reservation system, a customer can make online booking for a hotel, by specifying the accommodation requirements such as type of room (AC/Non-AC, One bed/two bed), total no of rooms, duration of stay. The system selects a suitable hotel as per customer's requirements. If a hotel is found then the availability of rooms in that hotel is checked. The charges are calculated for the selected requirement and these are acknowledged to the customer. If the customer is satisfactory about the selection made by the system, then he confirms the reservation.

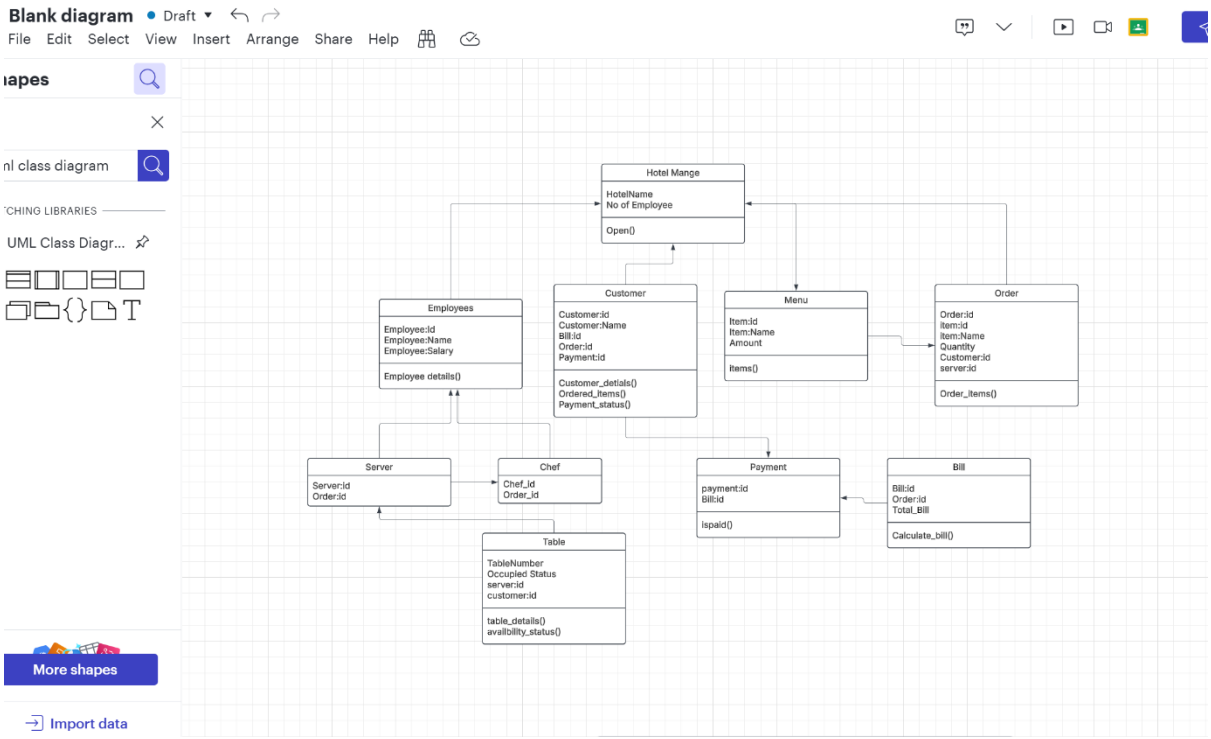
AIM:

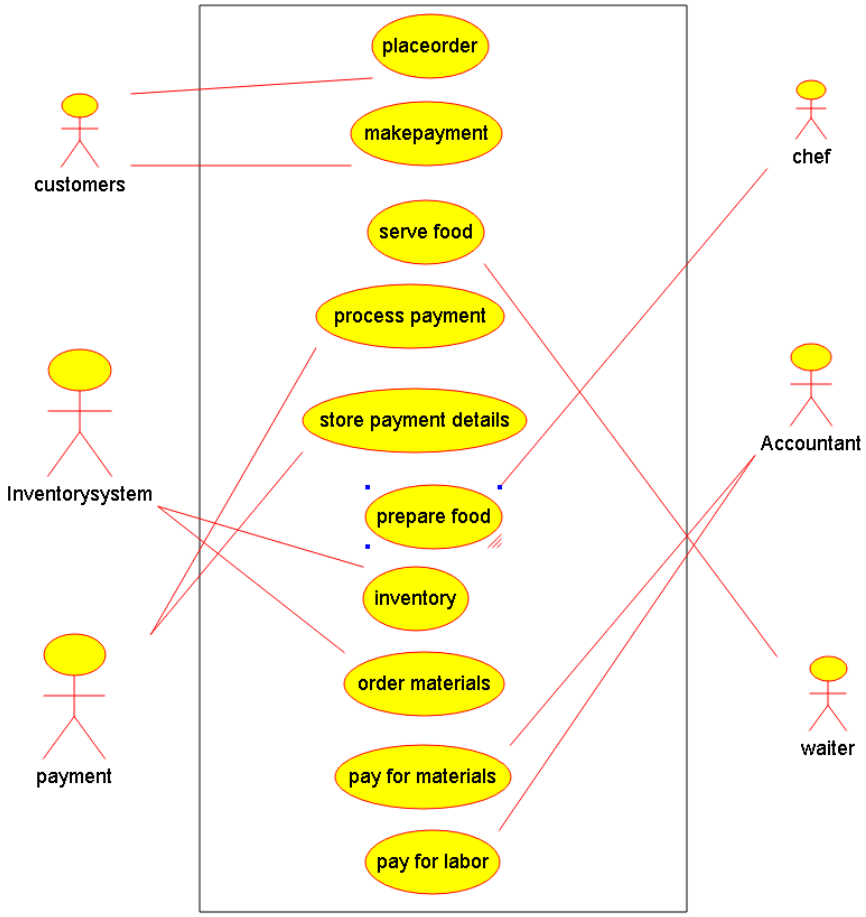
To design a UML diagram for a hotel reservation system that allows customers to book rooms online, checks room availability, calculates charges, and confirms reservations based on customer satisfaction.

Procedure:

1. Identify the main entities: Customer, Hotel, Room, Reservation, and Payment.
2. Define relationships: Customer books Reservation, Reservation includes Room, Hotel contains Room, and Payment is linked to Reservation.
3. Specify attributes: Room type, duration, charges, etc.
4. Model the flow: Customer selects requirements → System checks availability → Calculates charges → Confirms reservation.
5. Validate the system with use cases and sequence diagrams.

Output:





Result:

A UML Class Diagram for the Hotel Reservation System includes classes: Customer, Reservation, Room, and Payment. Relationships include Customer makes Reservation, Reservation includes Room, and Payment processes charges. Associations depict interactions for seamless room booking, availability checks, charge calculations, and reservation confirmations.

EXPERIMENT.NO: 2 coffe coffe day

Draw a coffe coffe day ordering system. A coffe coffee day shop vending machine dispenses coffee to customers. Customers order coffee by selecting a recipe from a set of recipes. Customers pay for the coffee using coins. Change is given back, if any, to the customers. The 'service assistant' loads ingredients (coffee powder, milk, sugar,) into the coffee machine. The 'service assistant' adds recipe by indicating the name of the coffee, the units of coffee powder, milk, sugar, water, chocolate to be added as well as the cost of the coffee.Develop the use case diagram for the specification above.

AIM:

To design a use case diagram for a Coffee Coffee Day ordering system where customers can order coffee, pay using coins, and receive change, while the service assistant manages recipes and ingredients.

Procedure:

1. Identify actors: Customer and Service Assistant.
2. Define use cases: Order Coffee, Pay for Coffee, Receive Change, Load Ingredients, Add Recipe, Edit Recipe, Delete Recipe.
3. Establish relationships: Customer interacts with ordering and payment, while Service Assistant manages recipes and ingredients.
4. Validate the flow: Customer selects a recipe → Pays → Receives coffee and change.
5. Ensure all functionalities are covered for both actors.

Output:

Result:

A Use Case Diagram for the Coffee Coffee Day ordering system includes actors: Customer and Service Assistant. The Customer can select a recipe, pay with coins, and receive coffee and change. The Service Assistant loads ingredients, adds recipes, and manages the machine. Use cases include "Order Coffee," "Pay for Coffee," "Dispense Coffee," "Load Ingredients," and "Add Recipe," illustrating interactions between actors and the system.

EXPERIMENT.NO: 3 Online Airline Reservation System

Make an Online Airline Reservation System. The activities of the Online Airline Reservation system are listed below user, admin, LOGIN, MANANGE CLASSES, MANANGE WAITING LIST, MANAGE HOLDS, MANAGE DEADLINES, LOGOUT, using this has a step-by-step process draw a CLASS diagram.

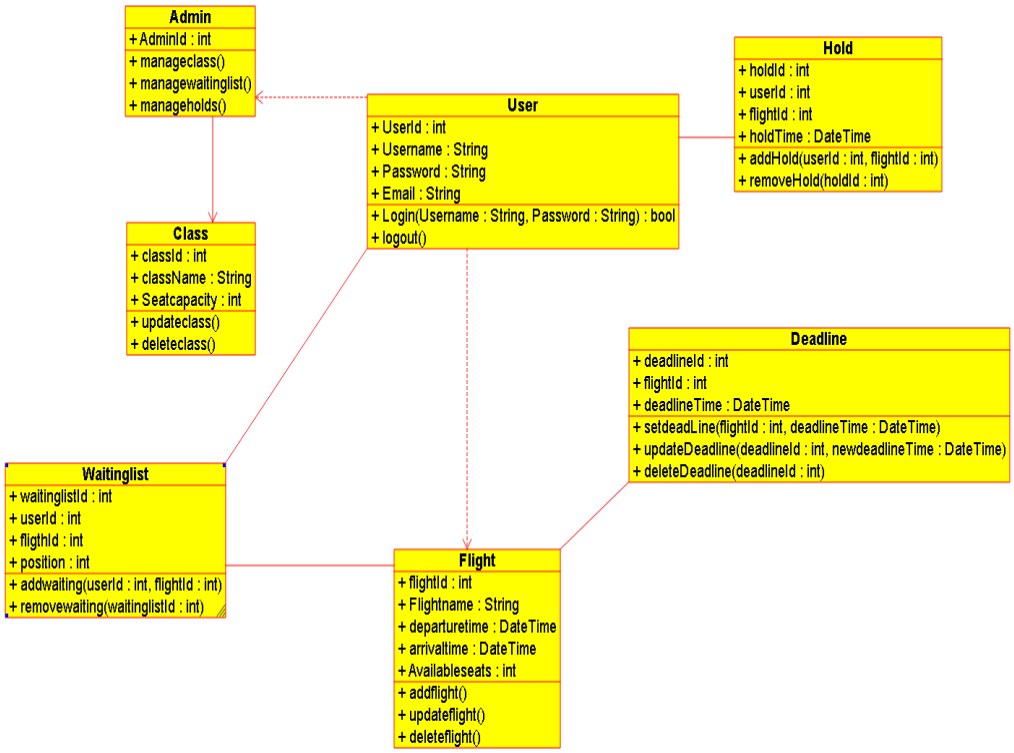
AIM:

To design a class diagram for an Online Airline Reservation System that includes functionalities for users and admins, such as managing classes, waiting lists, holds, deadlines, and login/logout.

Procedure:

1. Identify the main entities: **User**, **Admin**, **Flight**, **Reservation**, **WaitingList**, **Hold**, and **Deadline**.
2. Define attributes and methods for each class (e.g., User: login, logout; Admin: manage classes, manage waiting lists).
3. Establish relationships between classes (e.g., User makes Reservation, Admin manages Flight).
4. Validate the system flow: User logs in → Books flight → Admin manages reservations and deadlines.
5. Ensure all functionalities are represented in the class diagram.

Output:



Result:

A UML Class Diagram for the Online Airline Reservation System includes classes: User, Admin, Login, Class, WaitingList, Hold, and Deadline. Relationships include User and Admin inheriting from Login, with Admin managing Class, WaitingList, Hold, and Deadline. Methods include login(), logout(), manageClasses(), manageWaitingList(), manageHolds(), and manageDeadlines(). Associations depict interactions between entities for managing reservations, waiting lists, holds, and deadlines efficiently.

EXPERIMENT.NO: 4 ATM System

Draw a UML diagram for ATM System using CASE tool. The banking system allows a customer to access the financial transactions by ATM System, it has a step-by-step process describe the work of this process and elaborate the what are the work can do by customer, banking system, administrator and technicians with the ATM system.

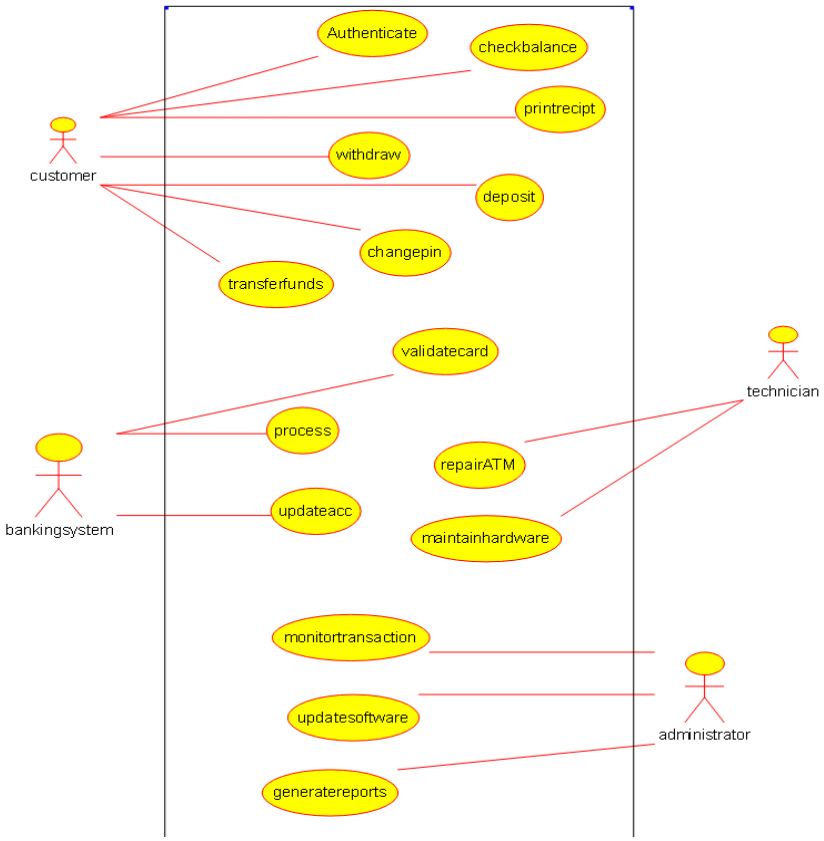
Aim:

To design a UML diagram for an ATM System using a CASE tool, illustrating the interactions between customers, the banking system, administrators, and technicians, and detailing the step-by-step process of financial transactions.

Procedure:

1. Identify the main entities: **Customer**, **Banking System**, **Administrator**, **Technician**, and **ATM**.
2. Define the functionalities: Customer performs transactions, Banking System validates and processes requests, Administrator manages accounts, and Technician maintains the ATM.
3. Establish relationships: Customer interacts with ATM, ATM communicates with Banking System, Administrator oversees operations, and Technician repairs/maintains the ATM.
4. Validate the flow: Customer inserts card → Enters PIN → Selects transaction → ATM processes request → Banking System validates → Transaction completes.
5. Ensure all roles and functionalities are represented in the UML diagram.

Output:



Result:

A UML Use Case Diagram for the ATM System includes actors: Customer, Banking System, Administrator, and Technician. The Customer can withdraw cash, check balance, deposit funds, and transfer money. The Banking System validates transactions, updates accounts, and manages security. Administrators oversee system operations and handle customer accounts, while Technicians maintain and repair the ATM hardware. The diagram illustrates interactions and processes for seamless financial transactions.

EXPERIMENT.NO: 5 food ordering system

Draw a UML diagram for a food ordering system Systems. The activities of the food ordering system are listed below. Receive the Customer food orders, Produce the customer ordered food, Serve the customer with their ordered food, collect payment from Customers, Store customer payment details, Order Raw Materials for food products, Pay for Raw Materials and Pay for Labour.

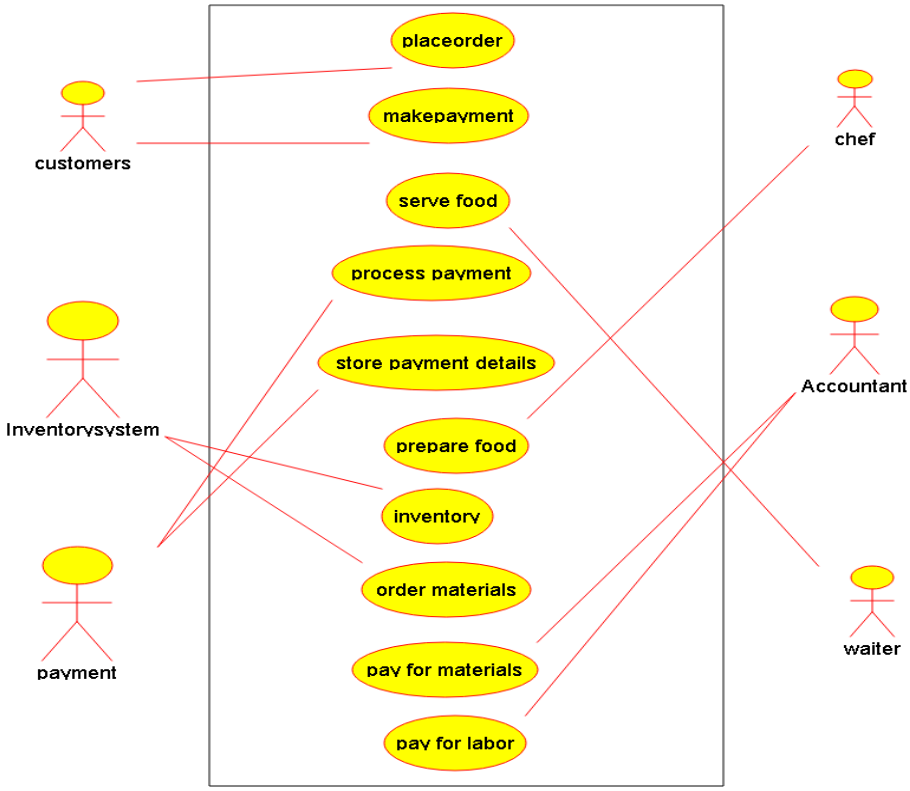
Aim:

To design a UML diagram for a Food Ordering System that captures the activities of receiving customer orders, producing food, serving customers, collecting payments, managing raw materials, and handling payments for labor and supplies.

Procedure:

1. Identify the main entities: Customer, Order, Food, Payment, RawMaterial, Supplier, and Labor.
2. Define the functionalities: Receive orders, produce food, serve food, collect payments, store payment details, order raw materials, and pay for labor/supplies.
3. Establish relationships: Customer places Order, Order includes Food, Payment is linked to Order, RawMaterial is ordered from Supplier, and Labor is paid for services.
4. Validate the flow: Customer places order → Food is prepared → Payment is collected → Raw materials are ordered and paid for.
5. Ensure all activities are represented in the UML diagram.

Ouput:



Result:

A UML Class Diagram for the food ordering system includes classes: Customer, Order, Food, Payment, Raw Material, and Labour. Relationships include Customer places Order, Order contains Food, Payment processes Customer payment, and System manages Raw Material and Labour. Associations depict interactions between entities for seamless operations.

EXPERIMENT.NO: 6 quiz system

Draw a Use case diagram to model for a quiz system. A user can request a quiz for the system. The system picks a set of questions from its database, and composes them together to make a quiz. It rates the user's answers and gives hints if the user requests it. In addition to users, we also have helpers who provide questions and hints. And also, administrators who must certify questions to make sure they are not too trivial, and that they are correct.

Aim:

To design a use case diagram for a Quiz System that models the interactions between users, helpers, and administrators, including functionalities like requesting quizzes, providing questions, certifying questions, and offering hints.

Procedure:

1. Identify the actors: **User**, **Helper**, and **Administrator**.
2. Define the use cases: Request Quiz, Take Quiz, Rate Answers, Provide Hints, Provide Questions, Certify Questions.
3. Establish relationships: User interacts with the system to take quizzes and request hints, Helper provides questions and hints, and Administrator certifies questions.
4. Validate the flow: User requests quiz → System composes quiz → User takes quiz → System rates answers → Helper provides hints → Administrator certifies questions.
5. Ensure all functionalities are represented in the use case diagram.

Output:

Result:

A Use Case Diagram for the quiz system includes actors: User, Helper, and Administrator. The User can request a quiz, receive hints, and get answers rated. Helpers provide questions and hints, while Administrators certify questions for correctness and appropriateness. The system composes quizzes, rates answers, and manages question certification.

EXPERIMENT.NO: 7 online purchasing system

Draw a UML diagram for online purchasing system. Provide top level use cases for a web customer making purchases online. Web customer actor uses some web site to make purchases online. Top level use cases are View Items, Make Purchase and Client Register.

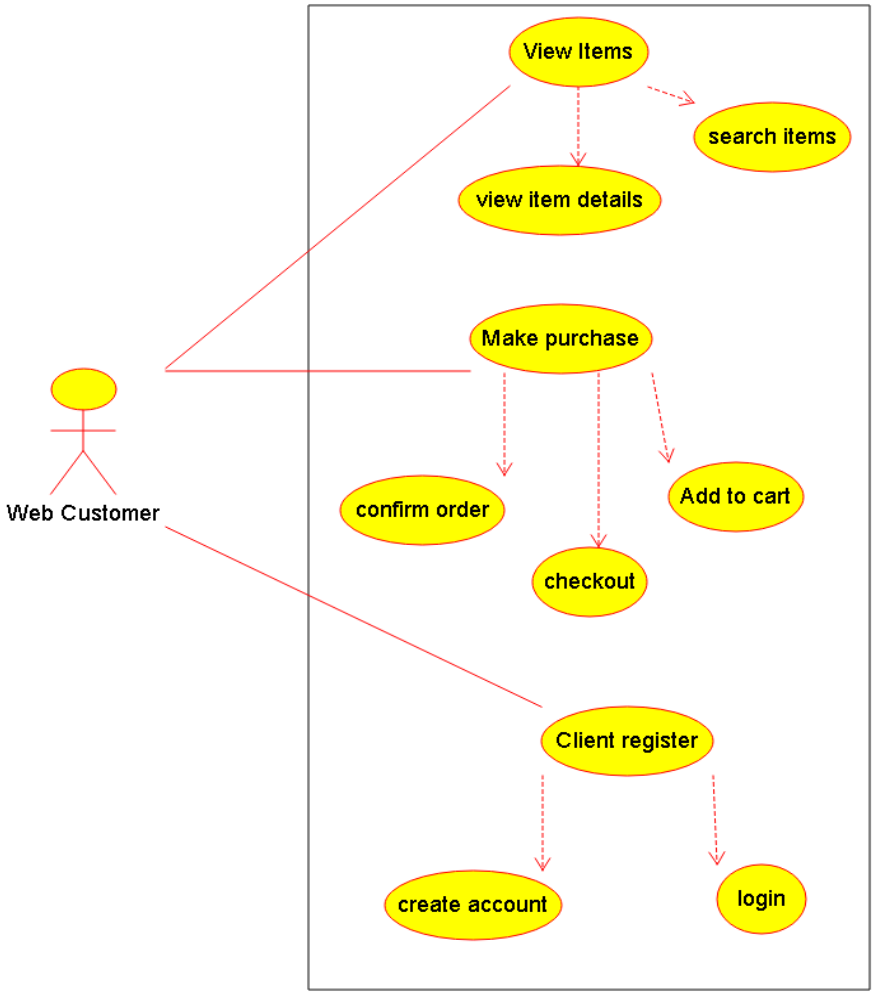
Aim:

To design a UML use case diagram for an Online Purchasing System that captures the top-level use cases for a web customer, including viewing items, making purchases, and registering as a client.

Procedure:

1. Identify the actor: **Web Customer**.
2. Define the top-level use cases: **View Items**, **Make Purchase**, and **Client Register**.
3. Establish relationships: Web Customer interacts with the system to view items, make purchases, and register.
4. Validate the flow: Customer registers → Views items → Makes a purchase.
5. Ensure all top-level functionalities are represented in the use case diagram.

Output:



Result:

In an online purchasing system, a web customer can perform several top-level use cases through the website. These primary use cases include viewing items, making purchases, and registering as a client. The "View Items" use case allows the customer to browse through available products.

EXPERIMENT.NO: 8 Hospital Management System

Describe major services (functionality) provided by a hospital's reception. Summary: Hospital Management System is a large system including several subsystems or modules providing variety of functions. Hospital Reception subsystem or module supports some of the many job duties of hospital receptionist. Receptionist schedules patient's appointments and admission to the hospital, collects information from patient upon patient's arrival and/or by phone. For the patient that will stay in the hospital ("inpatient") she or he should have a bed allotted in a ward. Receptionists might also receive patient's payments, record them in a database and provide receipts, file insurance claims and medical reports.

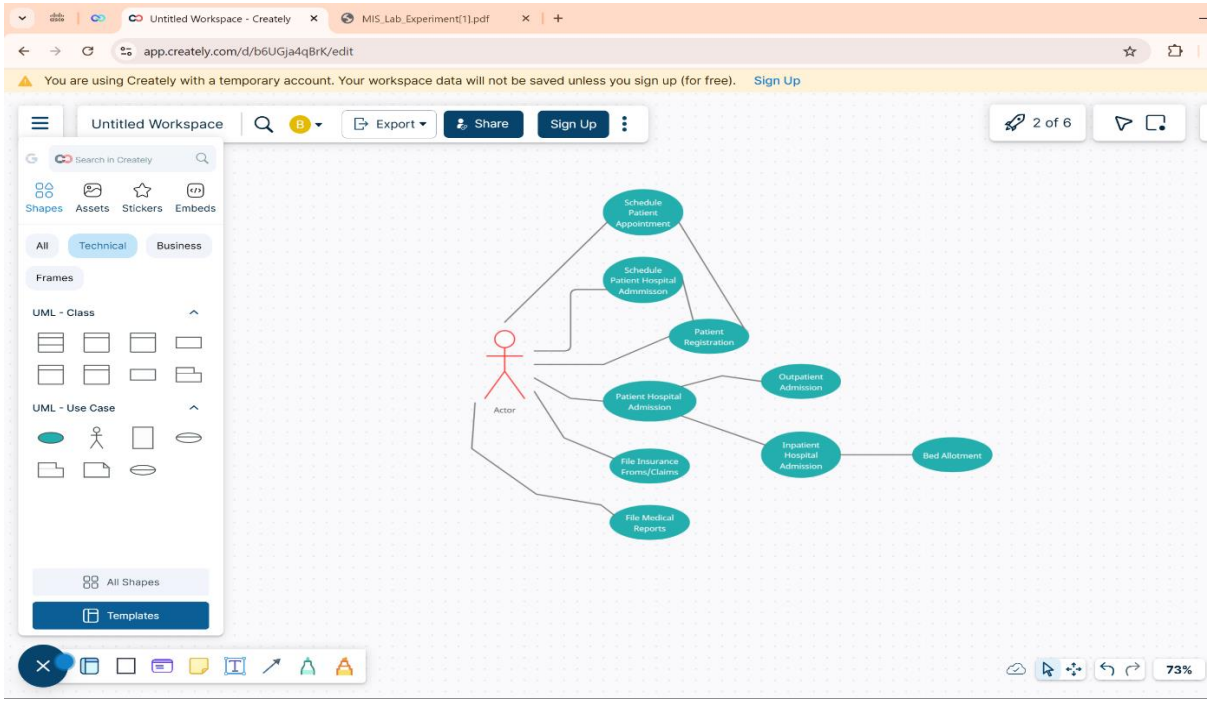
Aim:

To describe the major services (functionality) provided by a hospital's reception subsystem, focusing on tasks like scheduling appointments, managing patient information, handling payments, and coordinating inpatient admissions.

Procedure:

1. Identify the key functionalities: **Schedule Appointments**, **Manage Patient Information**, **Handle Payments**, **Allot Beds for Inpatients**, and **File Insurance Claims**.
2. Define the processes: Receptionist collects patient details, schedules appointments, admits inpatients, processes payments, and files insurance claims.
3. Validate the flow: Patient arrives → Receptionist collects information → Schedules appointment or admits inpatient → Handles payment and insurance.
4. Ensure all major services are covered, including both administrative and patient-facing tasks.
5. Highlight the role of the receptionist in coordinating between patients, doctors, and hospital systems.

Output:



Result:

The hospital reception handles several critical functions, including scheduling patient appointments and admissions, collecting patient information upon arrival or by phone, and allocating beds for inpatients. Receptionists also manage financial transactions by receiving and recording patient payments, providing payment receipts, and filing insurance claims and medical reports.

EXPERIMENT.NO: 9 Online security management system

A college has more than thousand security persons, who are instructed to give duties at different places within the campus. Additionally, they also maintain a routine, which contains all information, such as Date, Duty Start Time, Duty End Time, and Place. Most importantly, all the places are covered by at least one security person. If a security person takes leave, manual entry is done against that person. Finally, at the end of a month, the security persons get paid for their duties, while considering the number of leaves as well. You can see that the manual calculation/operation is a heavy task for the security manager. Therefore, the objective is to build an Online security management system using class diagram.

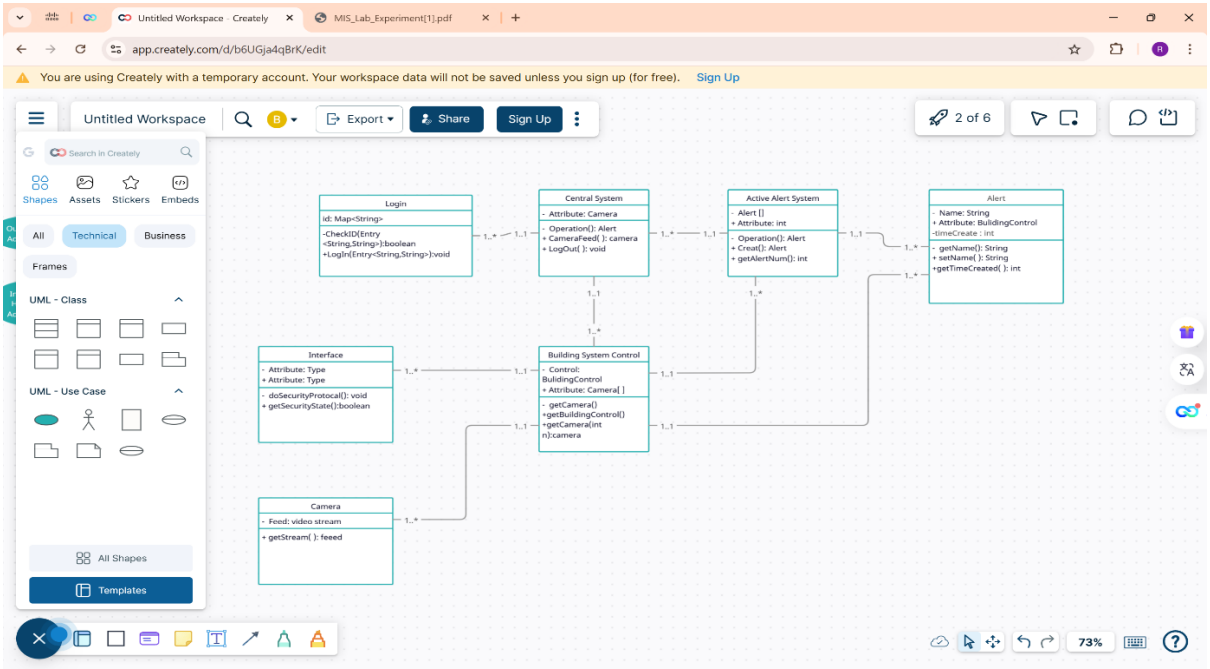
Aim:

To design a class diagram for an Online Security Management System that automates the scheduling, duty tracking, leave management, and payroll calculation for security personnel in a college campus.

Procedure:

1. Identify the main entities: **SecurityPerson**, **DutySchedule**, **LeaveRecord**, **Payroll**, and **SecurityManager**.
2. Define attributes and methods for each class (e.g., SecurityPerson: name, ID; DutySchedule: date, startTime, endTime, place).
3. Establish relationships: SecurityPerson is assigned to DutySchedule, LeaveRecord tracks absences, and Payroll calculates salary based on duties and leaves.
4. Validate the flow: SecurityPerson is assigned duties → Leaves are recorded → Payroll is calculated at month-end.
5. Ensure the system automates manual tasks like duty allocation, leave tracking, and salary calculation.

Output:



Result:

Above is the **class diagram** for the Online Security Management System, designed to automate scheduling, duty tracking, leave management, and payroll calculation for security personnel in a college.

EXPERIMENT.NO: 10 Library Management System

Draw a use case diagram for Library Management System is an automation system used to manage a library and the different resource management required in it like cataloguing of books, allowing check out and return of books, invoicing, user management, etc. The user can search for books details using few book properties (Book ID, Title, Author, and Publisher). Searching should return details about all the book copies that match the search query.

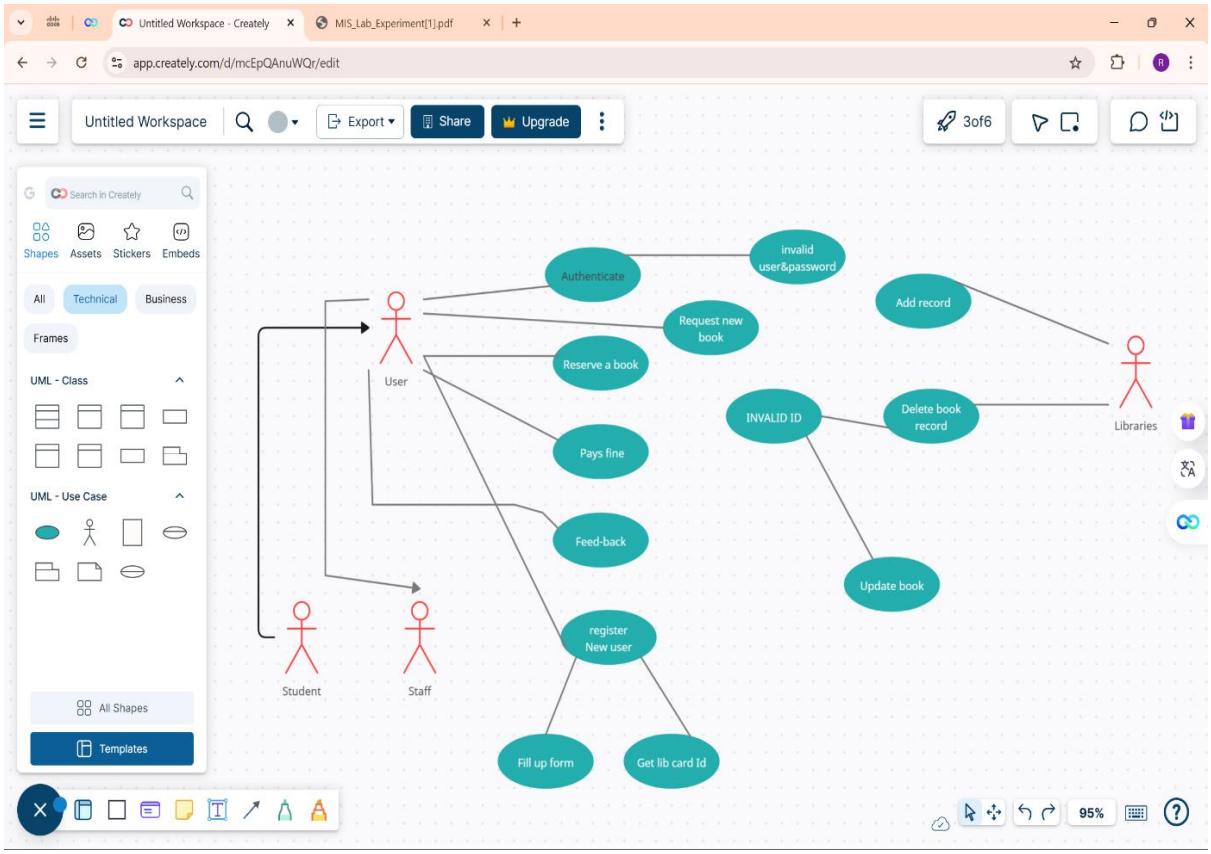
Aim:

To design a use case diagram for a Library Management System that automates library operations such as cataloguing books, managing checkouts and returns, invoicing, user management, and enabling book searches based on properties like Book ID, Title, Author, and Publisher.

Procedure:

1. Identify the actors: **Librarian**, **User**, and **System**.
2. Define the use cases: **Search Books**, **Check Out Books**, **Return Books**, **Manage Catalogue**, **Manage Users**, and **Generate Invoices**.
3. Establish relationships: User interacts with the system to search, check out, and return books; Librarian manages the catalogue, users, and invoices.
4. Validate the flow: User searches for books → Checks out or returns books → Librarian manages catalogue and users → System generates invoices.
5. Ensure all functionalities are represented in the use case diagram.

Output:



Result:

The use case diagram captures the interactions between the **User**, **Librarian**, and **System** in a Library Management System. It highlights key functionalities such as searching for books, checking out and returning books, managing the catalogue and users, and generating invoices.

VB program

DESIGN AND IMPLEMENTATION OF

EMPLOYEE INFORMATION SYSTEM

Form: employee details

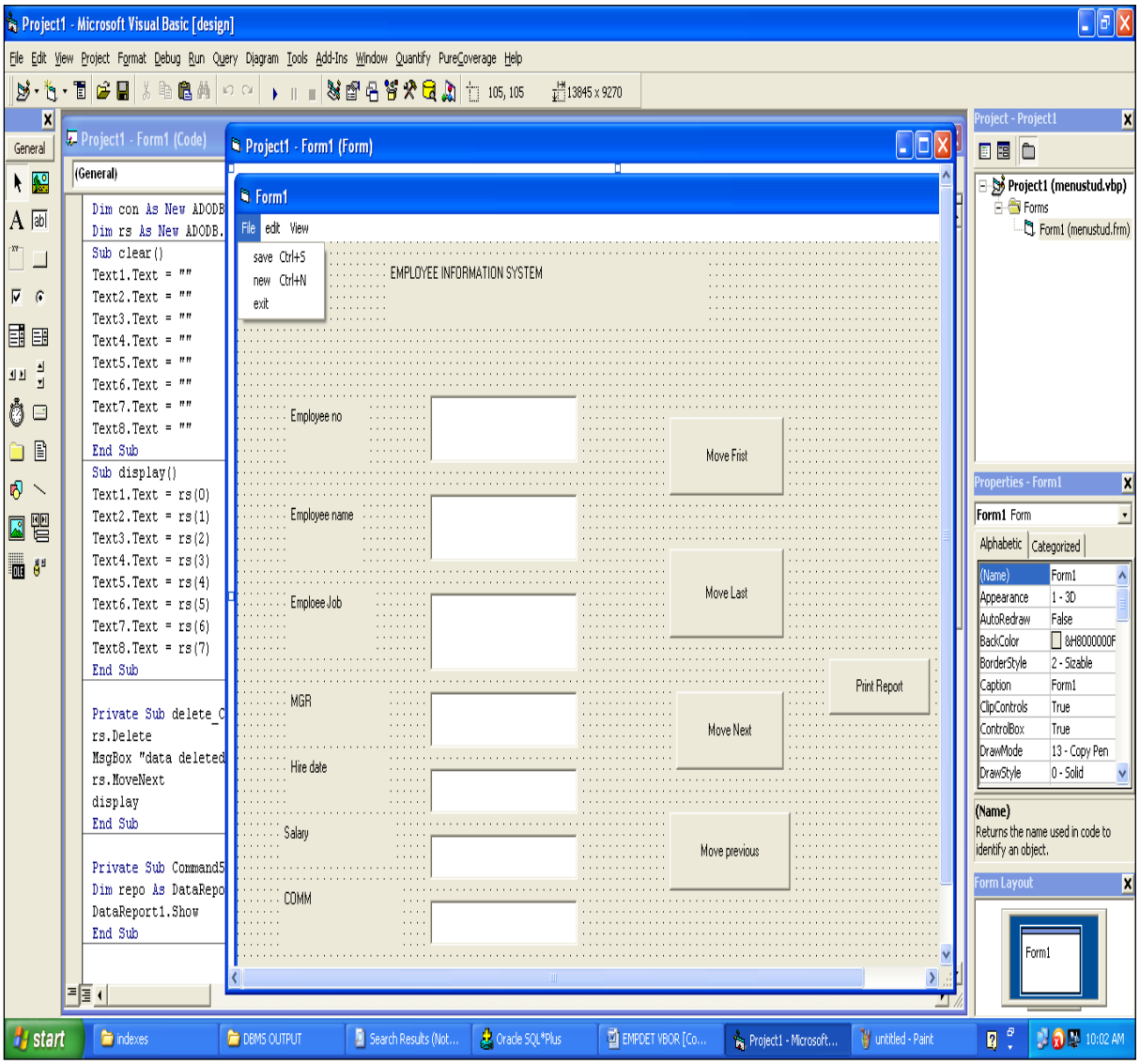
SQL> desc emp;

Name	Null?	Type
EMPNO	NOT NULL	NUMBER(4)
ENAME		VARCHAR2(10)
JOB		VARCHAR2(9)

MGR	**NUMBER(4)**
HIREDATE	**DATE**
SAL	**NUMBER(7,2)**
COMM	**NUMBER(7,2)**
DEPTNO	**NUMBER(2)**

Form design:

Input Form



CODING:

```
Dim con As New ADODB.Connection
Dim rs As New ADODB.Recordset

Sub clear()
Text1.Text = ""
Text2.Text = ""
Text3.Text = ""
Text4.Text = ""
Text5.Text = ""
Text6.Text = ""
Text7.Text = ""
Text8.Text = ""
End Sub

Sub display()
Text1.Text = rs(0)
Text2.Text = rs(1)
Text3.Text = rs(2)
Text4.Text = rs(3)
Text5.Text = rs(4)
Text6.Text = rs(5)
Text7.Text = rs(6)
Text8.Text = rs(7)
End Sub

Private Sub add_Click()
clear
Text1.SetFocus
End Sub
```

```
Private Sub delete_Click()
rs.delete
MsgBox "data deleted"
rs.MoveNext
display
End Sub

Private Sub exit_Click()
MsgBox ("thank u")
End
End Sub

Private Sub Form_Load()
con.Open "xxx", "scott", "tiger"
rs.Open "select * from emp", con, adOpenDynamic, adLockPessimistic
rs.MoveFirst
End Sub

Private Sub save_Click()
rs.MoveLast
rs.AddNew
rs(0) = Text1.Text
rs(1) = Text2.Text
rs(2) = Text3.Text
rs(3) = Text4.Text
rs(4) = Text5.Text
rs(5) = Text6.Text
rs(6) = Text7.Text
rs(7) = Text8.Text
rs.Update
```

```
display
MsgBox "Record Saved"
clear
End Sub

Private Sub view_Click()
Text1.Text = rs(0)
Text2.Text = rs(1)
Text3.Text = rs(2)
Text4.Text = rs(3)
Text5.Text = rs(4)
Text6.Text = rs(5)
Text7.Text = rs(6)
Text8.Text = rs(7)
End Sub

Private Sub movefirst_Click()
rs.movefirst
display
End Sub

Private Sub movelast_Click()
rs.movelast
display
End Sub

Private Sub moveprevious_Click()
rs.moveprevious
If rs.BOF = True Then
rs.movelast
display
```

```
Else
display
End If
End Sub

Private Sub movenext_Click()
rs.movenext
If rs.EOF = True Then
rs.movefirst
display
Else
display
End If
End Sub

Private Sub modify_Click()
rs(0) = Text1.Text
rs(1) = Text2.Text
rs(2) = Text3.Text
rs(3) = Text4.Text
rs(4) = Text5.Text
rs(5) = Text6.Text
rs(6) = Text7.Text
rs(7) = Text8.Text
rs.Update
MsgBox "record modified"
End Sub

Private Sub find_Click()
Dim w As Integer
w = InputBox("enter the employee number")
```

```
On Error GoTo er
rs.find "accno=" & w, adSearchForward
display
Exit Sub
er:
clear
MsgBox "record not found"
End Sub


Private Sub printreport_Click()
Dim repo As DataReport1
DataReport1.Show
End Sub
```

DESIGN AND IMPLEMENTATION OF STUDENT INFORMATION SYSTEM

Table description:

Form1: student details

Name	Null?	Type
ROLLNO		NUMBER(10)
NAME		VARCHAR2(10)
DEPT		VARCHAR2(3)
SEM		VARCHAR2(3)

Form1: mark details

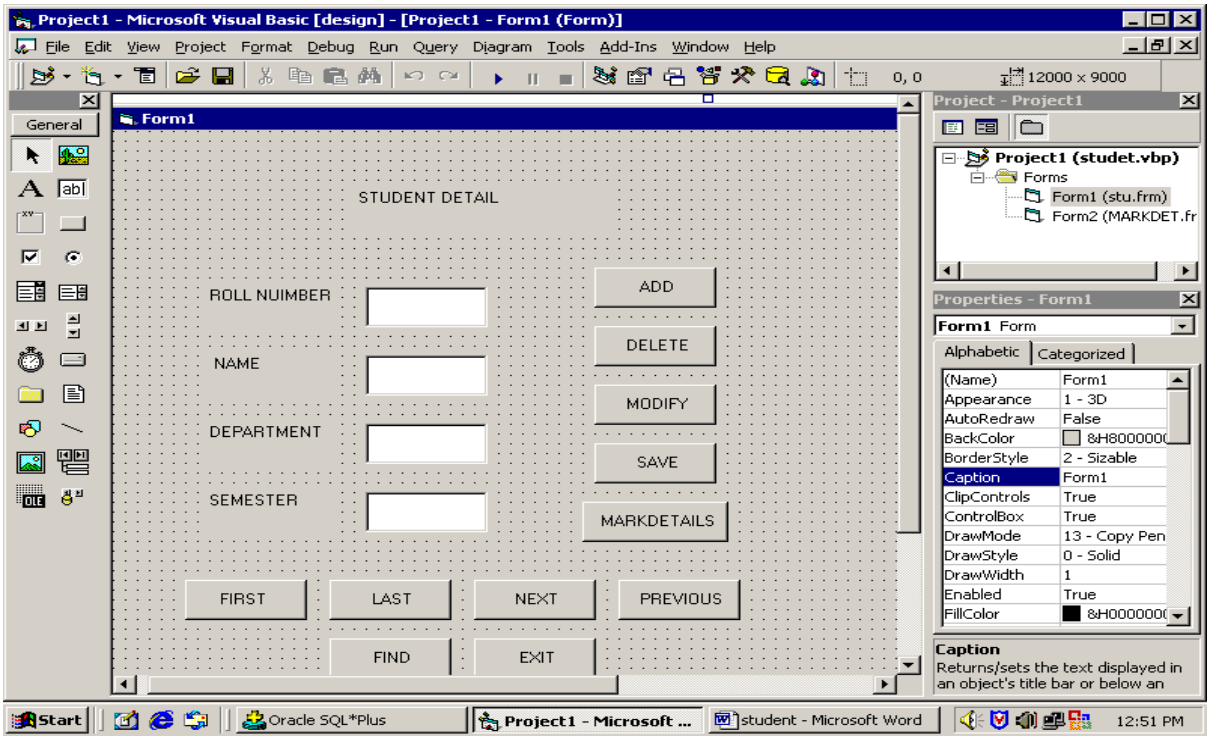
Name	Null?	Type

---------- -------- ----------

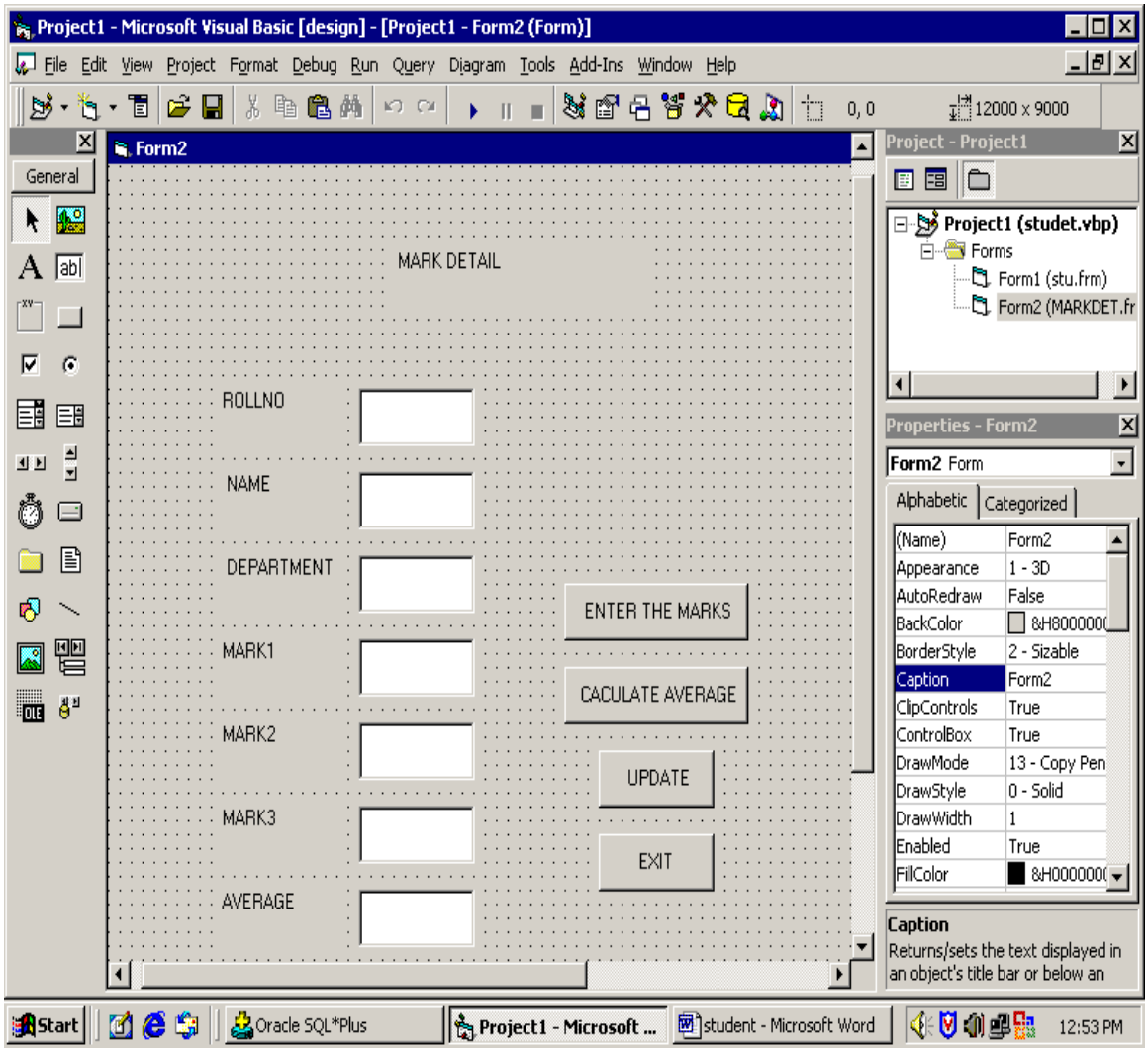
ROLLNO	**NUMBER(5)**
NAME	**VARCHAR2(10)**
DEPT	**VARCHAR2(4)**
MARK1	**NUMBER(3)**
MARK2	**NUMBER(3)**
MARK3	**NUMBER(3)**
AVERAGE	**NUMBER(7)**

Form design:

Form1: Student detail



Form2: Mark detail



Coding: FORM1

Dim con As New ADODB.Connection

Dim rs As New ADODB.Recordset

Sub clear()

Text1.Text = ""

Text2.Text = ""

Text3.Text = ""

Text4.Text = ""

End Sub

```
Sub display()
Text1.Text = rs(0)
Text2.Text = rs(1)
Text3.Text = rs(2)
Text4.Text = rs(3)
End Sub

Private Sub Command1_Click()
clear
Text1.SetFocus
End Sub

Private Sub Command11_Click()
Unload Me
End Sub

Private Sub Command2_Click()
rs.Delete
MsgBox "record deleted"
rs.MoveNext
display
End Sub

Private Sub Command3_Click()
rs(0) = Text1.Text
rs(1) = Text2.Text
rs(2) = Text3.Text
rs(3) = Text4.Text
rs.Update
MsgBox "record modified"
End Sub
```

```
Private Sub Command4_Click()
rs.MoveLast
rs.AddNew
rs(0) = Text1.Text
rs(1) = Text2.Text
rs(2) = Text3.Text
rs(3) = Text4.Text
rs.Update
display
MsgBox "Record updated"
clear
End Sub

Private Sub Command5_Click()
Form2.Show
End Sub

Private Sub Command6_Click()
rs.MoveFirst
display
End Sub

Private Sub Command7_Click()
rs.MoveLast
display
End Sub
```

```
Private Sub Command8_Click()
rs.MoveLast
If rs.EOF = True Then
rs.MoveFirst
display
Else
display
End If
End Sub

Private Sub Command9_Click()
rs.MovePrevious
If rs.BOF = True Then
rs.MoveLast
display
Else
display
End If
End Sub

Private Sub Command10_Click()
Dim w As Integer
w = InputBox("enter the rollno number")
On Error GoTo er
rs.Find "rollno=" & w, , adSearchForward
display
Exit Sub
er:
clear
MsgBox "record not found"
End Sub
```

```
Private Sub Form_Load()
con.Open "stu", "scott", "tiger"
rs.Open "select * from studen", con, adOpenDynamic, adLockPessimistic
rs.MoveFirst
End Sub

Coding: FORM2
Dim con, con1 As New ADODB.Connection
Dim rs, rs1 As New ADODB.Recordset

Sub clear()
Text4.Text = ""
Text5.Text = ""
Text6.Text = ""
Text7.Text = ""
End Sub

Sub display()
Text1.Text = Form1.Text1.Text
Text2.Text = Form1.Text2.Text
Text3.Text = Form1.Text3.Text
Text4.Enabled = False
Text5.Enabled = False
Text6.Enabled = False
Text7.Enabled = False
End Sub
```

```
Private Sub Command1_Click()
Text1.Text = Form1.Text1.Text
Text2.Text = Form1.Text2.Text
Text3.Text = Form1.Text3.Text
Text4.Enabled = True
Text5.Enabled = True
Text6.Enabled = True
Text4.SetFocus
End Sub

Private Sub Command2_Click()
Text7.Enabled = True
Text7.Text = (Val(Text4.Text) + Val(Text5.Text) + Val(Text6.Text)) / 3
End Sub

Private Sub Command3_Click()
Set rs1 = New ADODB.Recordset
rs1.Open "mark", con, adOpenDynamic, adLockPessimistic
rs1.AddNew
rs1(0) = Val(Text1.Text)
rs1(1) = Text2.Text
rs1(2) = Text3.Text
rs1(3) = Val(Text4.Text)
rs1(4) = Val(Text5.Text)
rs1(5) = Val(Text6.Text)
rs1(6) = Val(Text7.Text)
rs1.Update
MsgBox "updated"
rs1.Close
clear
Text1.Text = ""
```

Text2.Text = ""

Text3.Text = ""

End Sub

Private Sub Command4_Click()

Unload Me

End Sub

Private Sub Form_Load()

Set con = New ADODB.Connection

Set rs = New ADODB.Recordset

con.Open "stu", "scott", "tiger"

rs.Open "select * from studen", con, adOpenDynamic, adLockPessimistic

display

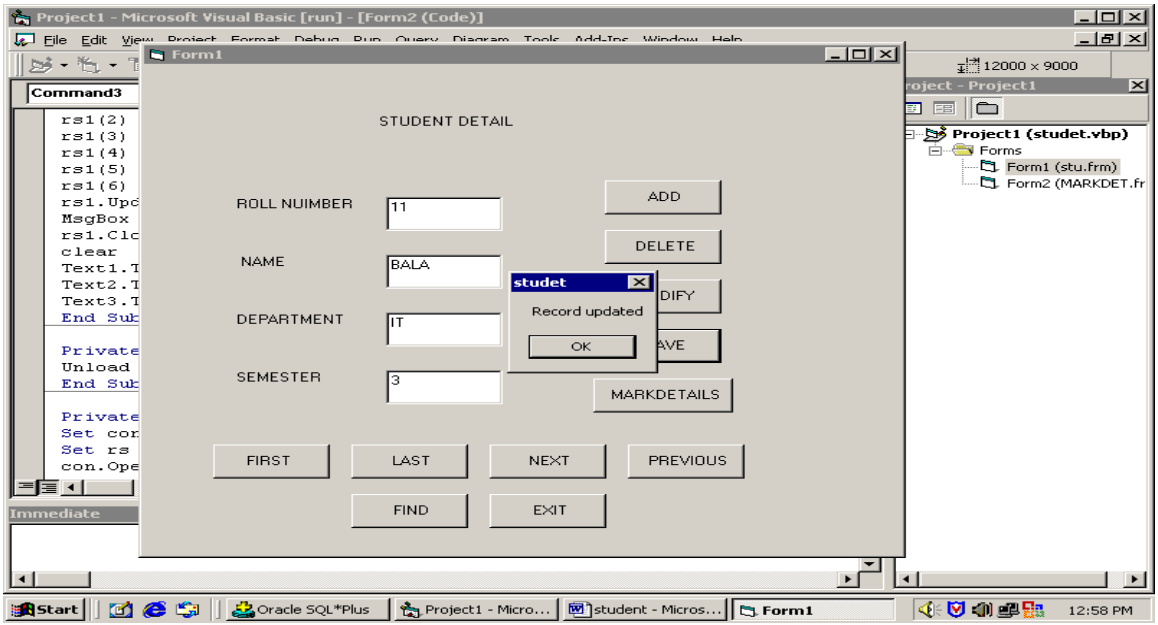
clear

End Sub

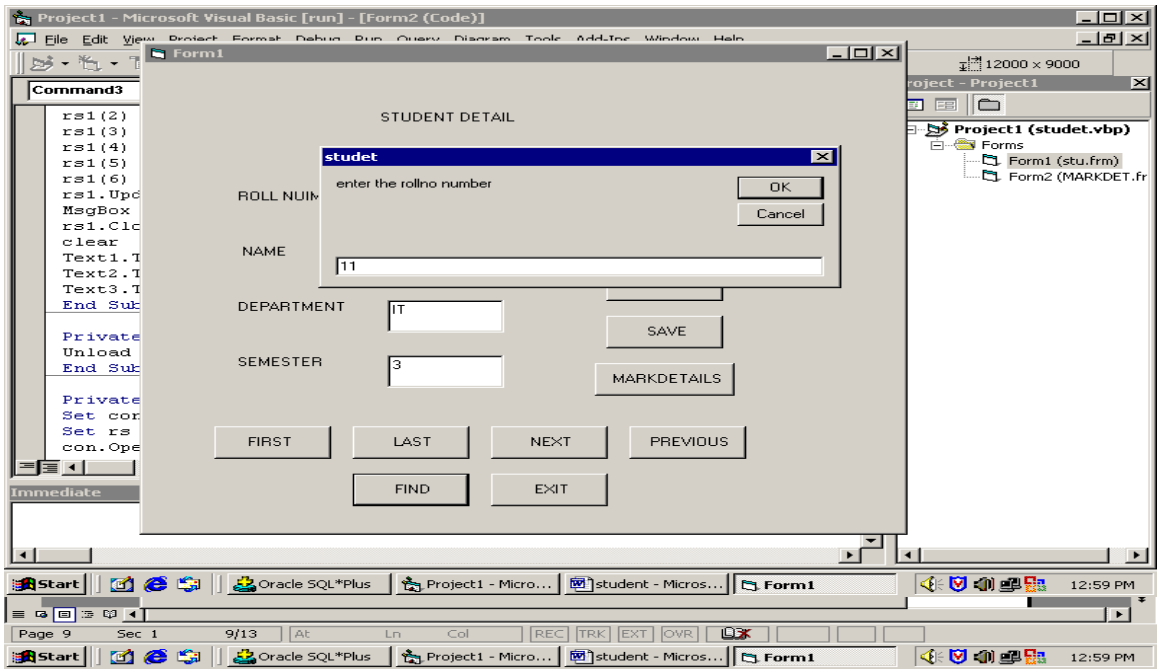
Output screen shots:

Student detail

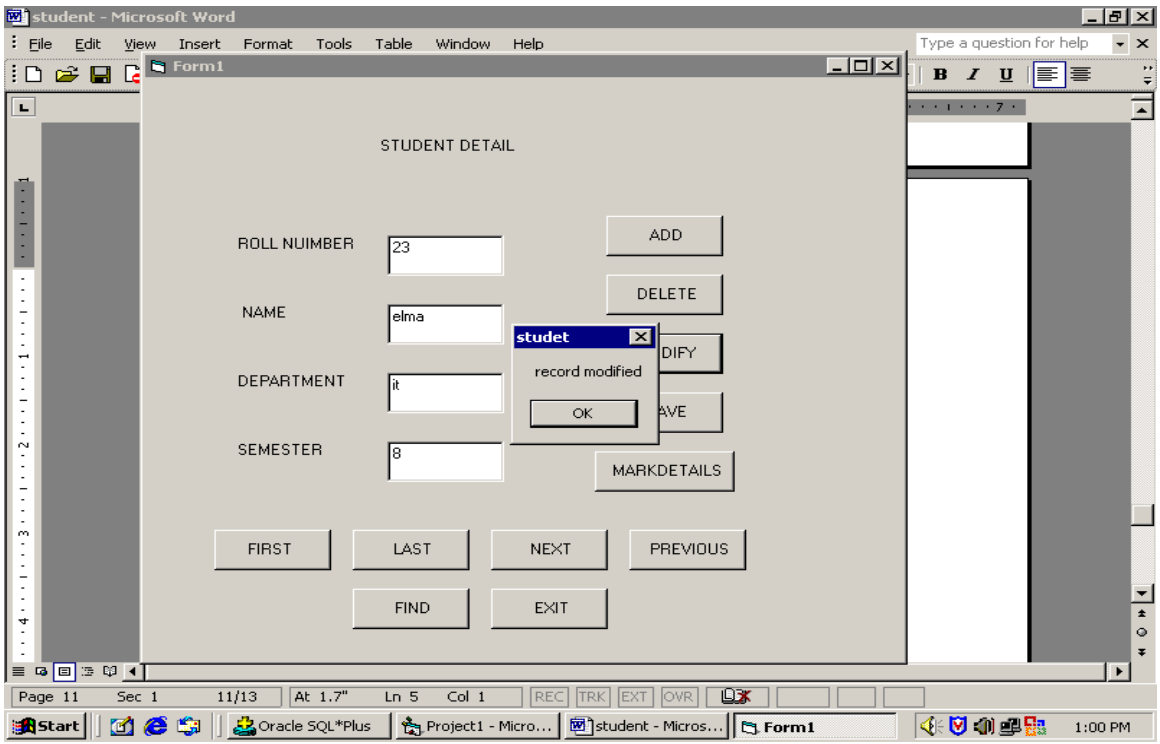
Save



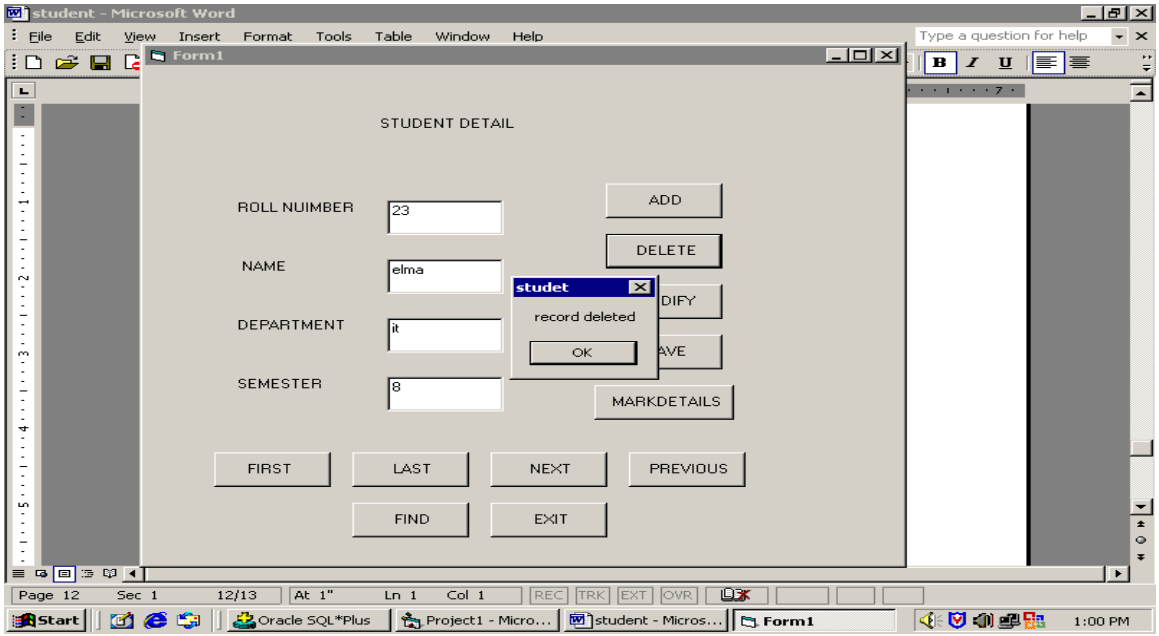
Find



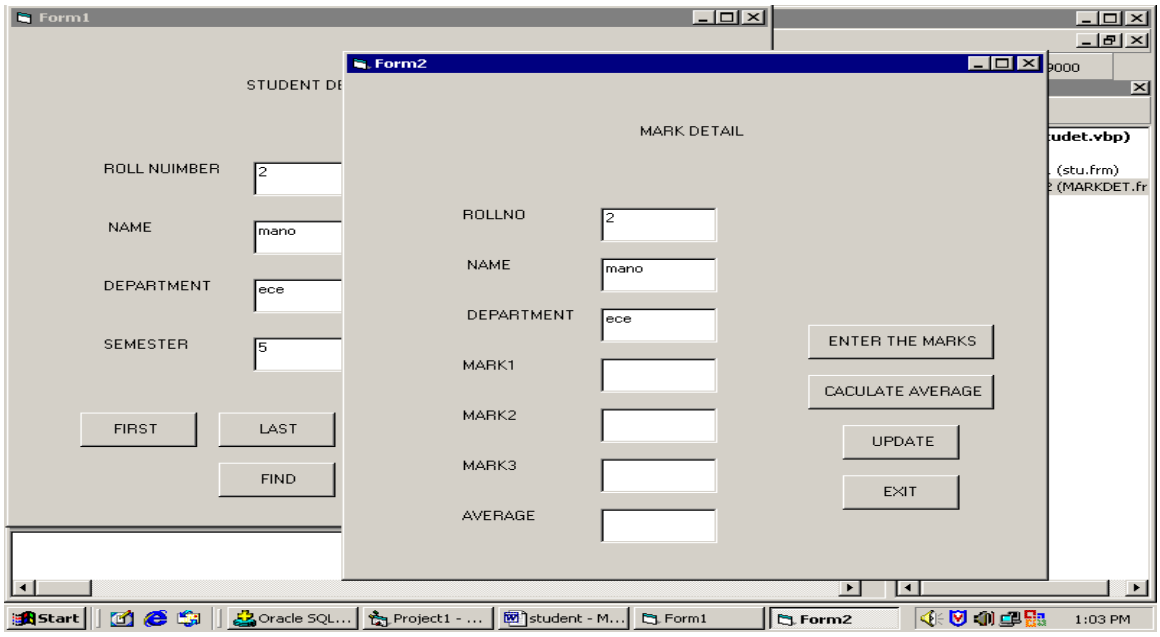
Modify



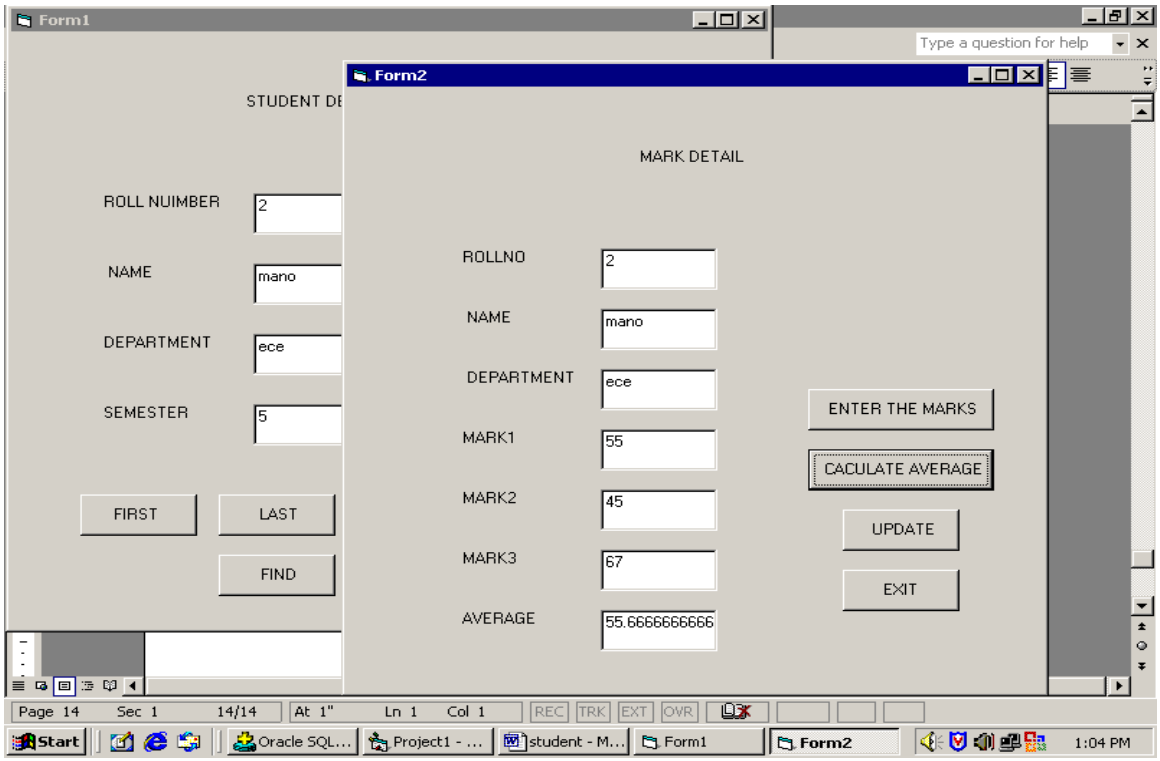
Delete



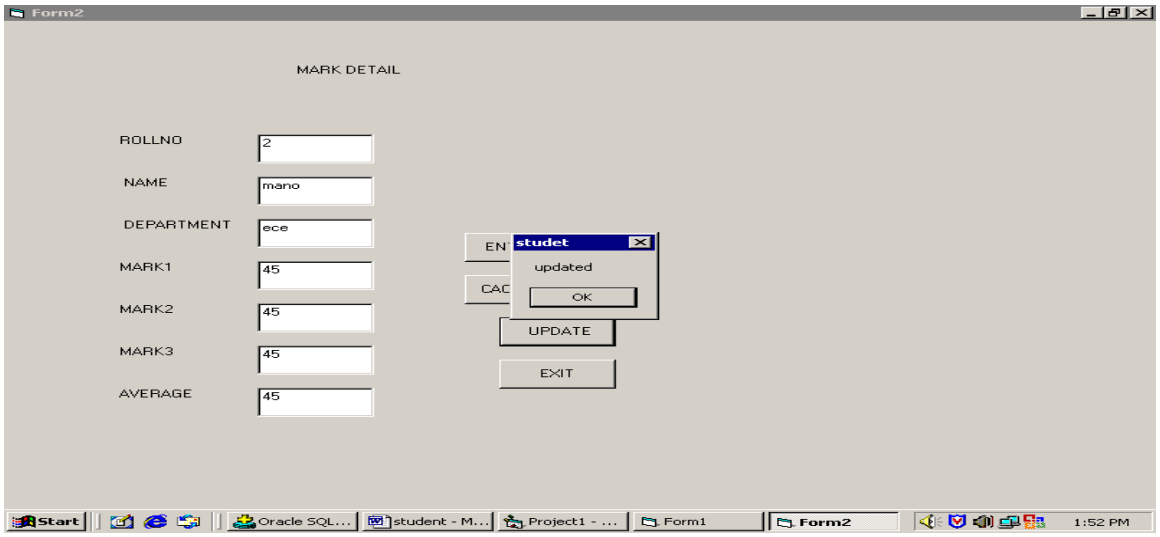
Mark detail screen shots



Calculate Average



Update



Result:

Thus the VB program was executed and employee information was retrieved successfully.

CODING:

```
Dim con As New ADODB.Connection
Dim rs As New ADODB.Recordset

Sub clear()
Text1.Text = ""
Text2.Text = ""
Text3.Text = ""
Text4.Text = ""
Text5.Text = ""
Text6.Text = ""
Text7.Text = ""
End Sub

Sub display()
Text1.Text = rs(0)
Text2.Text = rs(1)
Text3.Text = rs(2)
Text4.Text = rs(3)
Text5.Text = rs(4)
Text6.Text = rs(5)
Text7.Text = rs(6)
End Sub

Private Sub add_Click()
clear
Text1.SetFocus
End Sub

Private Sub delete_Click()
rs.delete
```

```
MsgBox "data deleted"
rs.MoveNext
display
End Sub

Private Sub exit_Click()
MsgBox ("thank u")
End
End Sub

Private Sub Form_Load()
con.Open "stud1", "scott", "tiger"
rs.Open "select * from student", con, adOpenDynamic, adLockPessimistic
rs.MoveFirst
End Sub

Private Sub save_Click()
rs.MoveLast
rs.AddNew
rs(0) = Text1.Text
rs(1) = Text2.Text
rs(2) = Text3.Text
rs(3) = Text4.Text
rs(4) = Text5.Text
rs(5) = Text6.Text
rs(6) = Text7.Text
rs.Update
display
MsgBox "Record Saved"
clear
End Sub
```

```
Private Sub view_Click()
Text1.Text = rs(0)
Text2.Text = rs(1)
Text3.Text = rs(2)
Text4.Text = rs(3)
Text5.Text = rs(4)
Text6.Text = rs(5)
Text7.Text = rs(6)
End Sub

Private Sub movefirst_Click()
rs.movefirst
display
End Sub

Private Sub movelast_Click()
rs.movelast
display
End Sub

Private Sub moveprevious_Click()
rs.moveprevious
If rs.BOF = True Then
rs.movelast
display
Else
display
End If
End Sub
```

```
Private Sub movenext_Click()
rs.movenext
If rs.EOF = True Then
rs.movefirst
display
Else
display
End If
End Sub

Private Sub modify_Click()
rs(0) = Text1.Text
rs(1) = Text2.Text
rs(2) = Text3.Text
rs(3) = Text4.Text
rs(4) = Text5.Text
rs(5) = Text6.Text
rs.Update
MsgBox "record modified"
End Sub

Private Sub find_Click()
Dim w As Integer
w = InputBox("enter the employee number")
On Error GoTo er
rs.find "accno=" & w, adSearchForward
display
Exit Sub
er:
clear
MsgBox "record not found"
```

```
End Sub
```

Optional

```
Private Sub deposit_Click()
Dim w As Integer
w = InputBox("enter the Deposit Amount")
Text4.Text = Val(Text4.Text) + w
MsgBox (Text4.Text)
rs(3) = Text4.Text
rs.update
MsgBox "record updated"
Text1.SetFocus
End Sub

Private Sub withdrawal_Click()
Dim w As Integer
w = InputBox("enter the Withdrawal Amount")
Text4.Text = Val(Text4.Text) - w
rs(3) = Text4.Text
rs.update
MsgBox "record updated"
End Sub

Private Sub printreport_Click()
Dim repo As DataReport1
DataReport1.Show
End Sub

Private Sub NextForm_Click()
Form2.show
End Sub
```